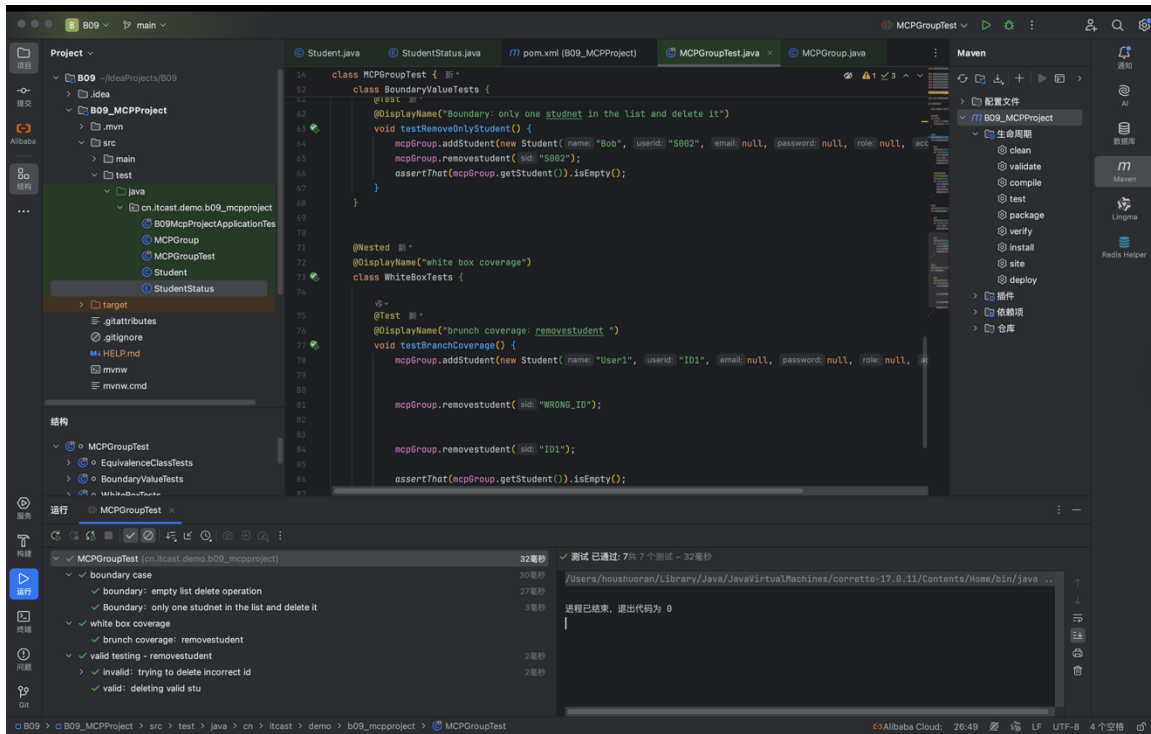




1. Fundamental Testing Strategy

- **Unit Testing:** We performed unit testing, which is the process of testing a single program or function in isolation. For this project, the "unit" is the MCPGroup class.
- **Test Oracles:** We used JUnit assertions as a **test oracle** to determine the expected output for each input, comparing actual results with the requirements specification.
- **Test Automation:** The testing process was automated to execute test cases, compare results, and generate reports efficiently.

2. Black-box Testing (Functional Testing)

This method generates test cases based on the requirements specification rather than the source code.

- **Equivalence-Class Testing:**

The input domain for removestudent(String sid) was divided into disjoint sub-domains.

Valid Equivalence Class: A student ID that exists in the current group (e.g., "S001").

Invalid Equivalence Class: A student ID that does not exist in the group

or is an empty string.

- **Boundary Value Testing:**

We focused on values near the borders of the input domain to catch errors where single keying mistakes might occur.

Case 1 (Empty List): Testing the system's behavior when trying to remove a student from a list with 0 elements.

Case 2 (Single Element): Testing a list with exactly one student to ensure the removal logic handles the minimum non-empty state correctly.

3. White-box Testing

This method generates test cases based on the internal source code and logic of the software.

- **Branch Testing:**

The objective was to ensure that every branch in the code is executed at least once.

In the `removestudent` method, we covered the branch where the `userid.equals(sid)` condition is **True** (student found and removed) and the branch where it is **False** (no match found).

- **Statement Testing:**

We ensured that every statement within the `MCPGroup` methods (like adding or getting students) was executed at least once.

4. Verification and Validation (V & V)

- **Verification:** We checked if the system met the specifications (e.g., does `addStudent` return a boolean as specified?).
- **Validation:** We ensured the system met the user's requirements (e.g., does it actually help a user manage a group of students as expected?).